Manual Extraction and Reconstruction of the System Boot Key (SysKey)

Objective

The purpose of this procedure is to manually extract the four hidden fragments of the Windows System Boot Key (JD, Skew1, GBG, Data) from the SYSTEM registry hive. We will utilize **Registry Explorer** to identify logical offsets and a **Hex Editor** to extract the physical data, finally reconstructing the key using the standard Windows scrambling algorithm.

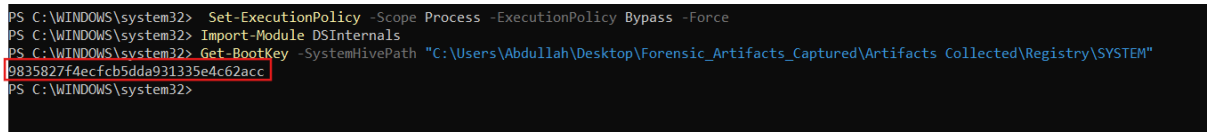
Phase 1: Preliminary Verification (Automated)

Before performing the manual extraction, we utilized the automated tool **DSInternals** to retrieve the System Boot Key. This serves as a "control" value to verify the accuracy of our subsequent manual reconstruction.

Execution:

We ran the following PowerShell command against the captured registry hives:

Get-BootKey -SystemHivePath "\path\to\SYSTEM\Hive\" **Result:**



The tool successfully extracted the following Boot Key:

9835827f4ecfcb5dda931335e4c62acc

This value will serve as our reference point. Our goal in the following phases is to manually reconstruct this exact key from the raw hex bytes.

Phase 2: Logical Offset Discovery (Registry Explorer)

Unlike standard registry values which appear in the main "Values" panel, the SysKey fragments are hidden in the **Key Metadata**. To locate them, we must access the technical details of the specific registry keys.

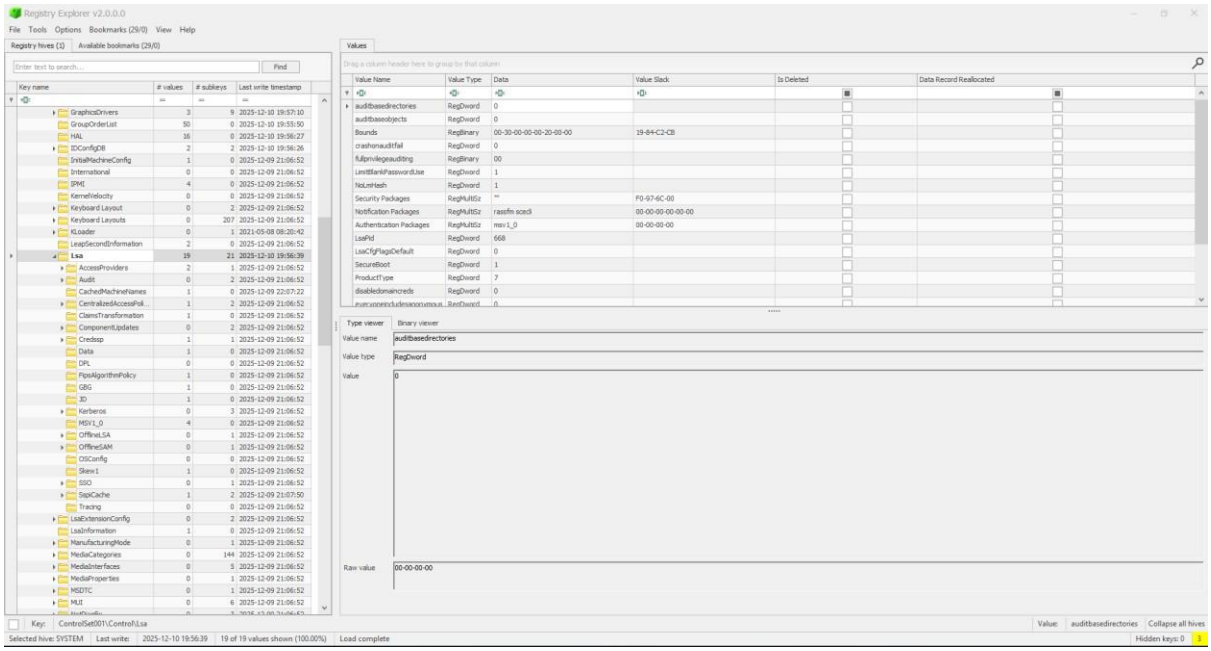
Step 1: Locating the Relative Offset

1. **Load the Hive:** Open **Registry Explorer** and load your captured SYSTEM hive.
2. **Navigate to the LSA Key:** Browse to the following path:

ROOT \ ControlSet001 \ Control \ Lsa

3. **Identify Target Folders:** Within the Lsa key, locate the following four sub-keys:
    - JD o Skew1 o GBG o Data

(Note: All these folders are located within the same Lsa parent folder.)



4. **Extracting Technical Details (Demonstration with "JD"):**
    - Right-click on the **JD** folder. o Select **Technical Details** from the context menu. o In the pop-up window, check the box or tab labeled **"Full details as text"**.
    - Scroll down to find the line labeled: Class cell index: 0xD1710 o *(Note: This value will vary for different folders).*
5. **Record the Offset:** Copy this value. This is your **Relative Offset**.
6. **Repeat:** Perform this process for the Skew1, GBG, and Data folders and record their respective Class Cell Indexes.

Phase 3: Calculating Physical Addresses

The offsets provided by Registry Explorer are **relative** to the start of the hive bins. To find the exact location of this data on the physical disk (viewable in a Hex Editor), we need to add the size of the Registry Hive Header.

Formula:

Relative Offset + 0x1000 (File Header) = **Physical Address Calculations:**

Applying this formula to our findings:

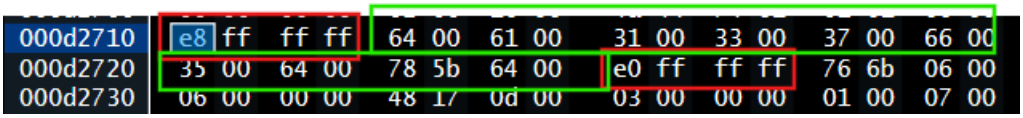
- **JD:** 0xD1710 + 0x1000 = **0xD2710**
- **Skew1:** 0xD1D90 + 0x1000 = **0xD2D90**
- **GBG:** 0xD1AA0 + 0x1000 = **0xD2AA0**
- **Data:** 0xD1E80 + 0x1000 = **0xD2E80**

Phase 4: Physical Extraction (Hex Editor)

We proceeded to navigate to the calculated physical addresses in the Hex Editor to reveal and extract the raw data structures.

1. **Visual Analysis (Demonstration with "JD")**

Navigating to **0xD2710** in the Hex Editor reveals the raw data structure. Below is the breakdown of the color-coded regions in the image.



- **Red Box - The Cell Header** o **Hex Value:** e8 ff ff ff o **Analysis:** This represents the **Cell Size**. In the Windows Registry, a negative number indicates an "Allocated" (active) cell. This tells the system that a valid data block begins here.
- **Green Box- Class Name (The Key Fragment)** o **Hex Value:** 64 00 61 00 31 00 33 00 37 00 66 00 35 00 64 00
    - **Analysis:** These 16 bytes are the hidden Class Name defined in Registry Explorer.
    - **Decoded:** d.a.1.3.7.f.5.d which is then converted to **da137f5d**.
    - *(Note: The green box in the image highlights the full data block, including padding bytes, but the first 16 bytes contain the actual key fragment).*
- **Red Box - Next Cell** o **Hex Value:** e0 ff ff ff o **Analysis:** This is the header for the **Next Cell** immediately following our data. The presence of this

new header confirms that our "Class Data" block has ended and we have captured the complete value without truncation.

2. **Extraction of Remaining Keys**

Using the physical addresses calculated in Phase 3, we extracted the remaining fragments:

Extraction of "Skew1"

- **Offset:** 0xD2D90
- **Class Name bytes:** 38 00 32 00 33 00 35 00 39 00 33 00 63 00 63 00
- **Decoded ASCII: 823593cc Extraction of "GBG"**
- **Offset:** 0xD2AA0
- **Class Name bytes:** 39 00 38 00 63 00 66 00 63 00 36 00 34 00 65 00
- **Decoded ASCII: 98cfc64e Extraction of "Data"**
- **Offset:** 0xD2E80
- **Class Name bytes:** 33 00 35 00 63 00 62 00 65 00 34 00 32 00 61 00
- **Decoded ASCII: 35cbe42a**

Phase 5: Reconstructing the Boot Key

To obtain the final SysKey, we processed the four extracted fragments using the standard Windows scrambling algorithm.

Step 1: Concatenation (Endianness Conversion)

We converted each 8-byte string from Little Endian to Big Endian and concatenated them:

- **JD:** da137f5d is converted to **5d 7f 13 da**
- **Skew1:** 823593cc is converted to **cc 93 35 82**
- **GBG:** 98cfc64e is converted to **4e c6 cf 98**
- **Data:** 35cbe42a is converted to **2a e4 cb 35 Raw Scrambled Key:**

5d 7f 13 da cc 93 35 82 4e c6 cf 98 2a e4 cb 35

Step 2: The SysKey Scramble (Permutation Logic)

First, we took our four extracted keys (JD, Skew1, GBG, Data), reversed them, and lined them up in a single row. This created a pool of 16 bytes. We assign a **"Position Number"** (0 to 15) to each byte so we can identify them. **Our Raw Input Pool:**

5d 7f 13 da cc 93 35 82 4e c6 cf 98 2a e4 cb 35

- **Position 0** holds 5d
- **Position 6** holds 35 • **Position 11** holds 98
- ...and so on.

The Constant Matrix:

Windows uses a specific set of constant numbers to tell us how to scramble this data. Think of this matrix as a list of instructions for filling 16 empty slots. **Matrix:** [11, 6, 7, 1, 8, 10, 14, 0, 3, 5, 2, 15, 13, 9, 12, 4]

Step 3: Execution

To create the final System Key, we go through the Matrix one number at a time. The number tells us **"Go to this Position in the Input Pool and copy that byte here."**

1. **Slot 1:** The Matrix says **"11"**.
    - *Action:* We go to Position 11 in our pool.
    - *Result:* We find the byte **98**. This becomes the first byte of our final key.
2. **Slot 2:** The Matrix says **"6"**.
    - *Action:* We go to Position 6 in our pool.
    - *Result:* We find the byte **35**. This becomes the second byte.
3. **Slot 3:** The Matrix says **"7"**.
    - *Action:* We go to Position 7 in our pool.
    - *Result:* We find the byte **82**. This becomes the third byte.
4. **Slot 4:** The Matrix says **"1"**.

- *Action:* We go to Position 1 in our pool.
- *Result:* We find the byte **7f**. This becomes the fourth byte.

By repeating this process for all 16 numbers in the matrix, the scrambled raw data is transformed into the valid, orderly SysKey.

Final Reconstructed SysKey:

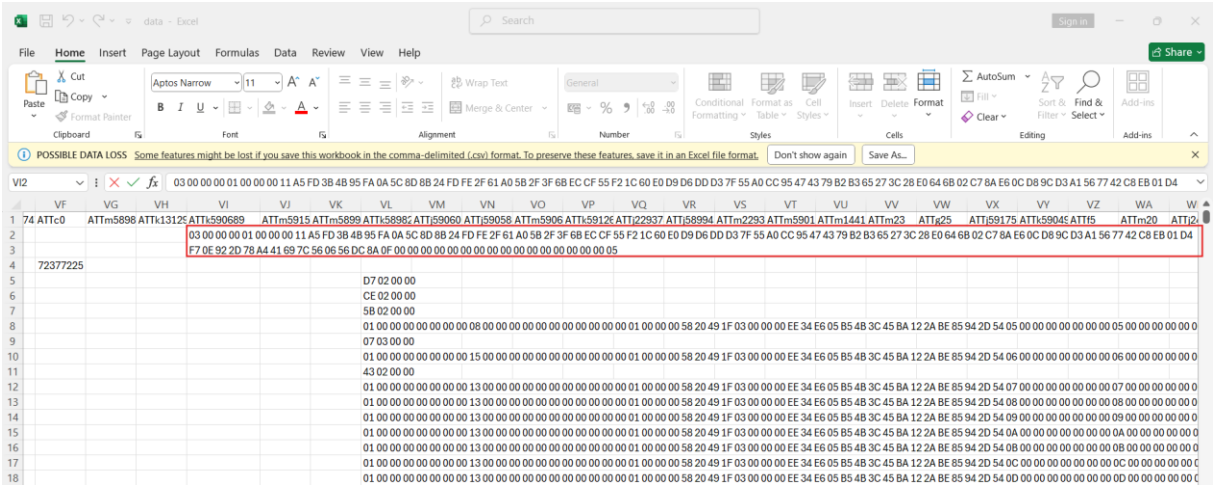
9835827f4ecfcb5dda931335e4c62acc

Conclusion

We have successfully manually extracted and verified the System Boot Key. This key matches the reference key obtained via DSInternals in Phase 1 and is now ready to be used for decrypting the Password Encryption Key (PEK) from the NTDS.dit database.

STEP 2: Decrypting the Password Encryption Key (PEK)

The PEK is stored in the ntds.dit database within the datatable. It is located in the row representing the Domain Root object (which was found at DNT 2043 in this instance however it can be at any instance) under the column ATTk590689. We opened Ntds.dit file in ESE Database viewer and exported the datatable as csv file which was too large to navigate on ESE database.Then we navigate to the coloumn ATTk590689 and finded this raw hex.



The raw hex string identified was: 03 00 00 00 01 00 00 00 11 A5 FD 3B 4B 95 FA 0A 5C 8D 8B 24 FD FE 2F 61 A0 5B 2F 3F 6B EC CF 55 F2 1C 60 E0 D9 D6 DD D3 7F 55 A0 CC 95 47 43 79 B2 B3 65 27 3C 28 E0 64 6B 02 C7 8A E6 0C D8 9C D3 A1 56 77 42 C8 EB 01 D4 F7 0E 92 2D 78 A4 41 69 7C 56 06 56 DC 8A

2. Parsing the Encryption Header

Before decryption, the payload must be split into its functional components based on the Active Directory encryption standards.

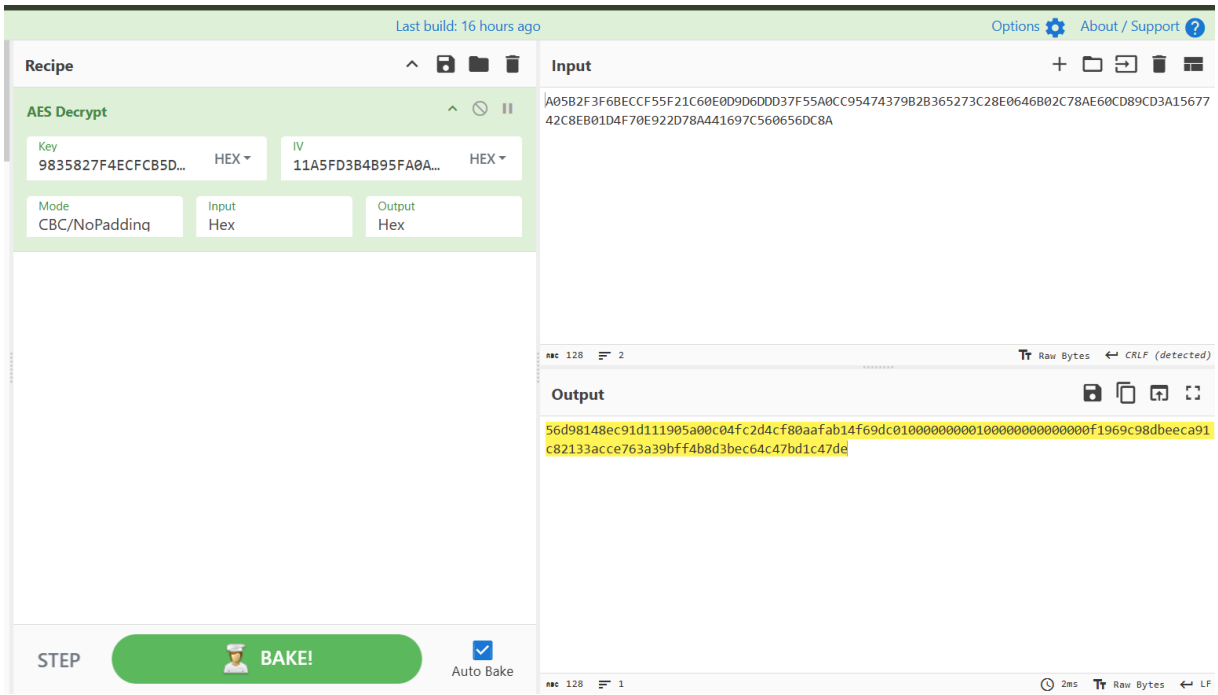
- **Version Header (Bytes 0-8):** 03 00 00 00 01 00 00 00. The 03 indicates that the domain is using **AES-128** encryption.
- **Initialization Vector (IV) (Bytes 8-24):** 11 A5 FD 3B 4B 95 FA 0A 5C 8D 8B 24 FD FE 2F 61. This 16-byte value is required for the AES-CBC mode.
- **Ciphertext (Bytes 24+):** A0 5B 2F 3F 6B EC CF 55 F2 1C 60 E0 D9 D6 DD D3 7F 55 A0 CC 95 47 43 79 B2 B3 65 27 3C 28 E0 64 6B 02 C7 8A E6 0C D8 9C D3 A1 56 77 42 C8 EB 01 D4 F7 0E 92 2D 78 A4 41 69 7C 56 06 56 DC 8A

  This is the encrypted block containing the PEK.

3. Carving the actual PEK:

To carve the actual PEK from this raw text, we use the tool CyberChef to decrypt the AES ciphertext. We supplied the following parameters into the "AES Decrypt" recipe:

- **Input:** A0 5B 2F 3F 6B EC CF 55 F2 1C 60 E0 D9 D6 DD D3 7F 55 A0 CC 95 47 43 79 B2 B3 65 27 3C 28 E0 64 6B 02 C7 8A E6 0C D8 9C D3 A1 56 77 42 C8 EB 01 D4 F7 0E 92 2D 78 A4 41 69 7C 56 06 56 DC 8A
- **Key:** 9835827f4ecfcb5dda931335e4c62acc (The previously extracted Syskey/Boot Key)
- **IV(Initialization Vector):** 11A5FD3B4B95FA0A5C8D8B24FDFE2F61 (Parsed from the payload above)
- **Mode:** CBC/No Padding
- **Padding:** NoPadding (Crucial for bypassing standard PKCS#7 expectations on raw Active Directory structures)



The successful AES decryption operation yielded the following raw hex output:
56 d9 81 48 ec 91 d1 11 90 5a 00 c0 4f c2 d4 cf 80 aa fa b1 4f 69 dc 01 00 00 00 00 01 00 00 00 00 00 00 00 00 f1 96 9c 98 db ee ca 91 c8 21 33 ac ce 76 3a 39 bf f4 b8 d3 be c6 4c 47 bd 1c 47 de

However, this output does not immediately represent a usable raw cryptographic key. In Active Directory architecture, the decrypted payload is a serialized C-style data structure representing the internal PEK List. To extract the actual symmetric key, we must carve the blob based on its documented byte offsets.
By analyzing the decrypted block, we identified the following internal components:

- **Key GUID (Bytes 0-15):**
  56 d9 81 48 ec 91 d1 11 90 5a 00 c0 4f c2 d4 cf
  This 16-byte segment serves as the globally unique identifier for this specific version of the encryption key within the domain context.
- **Creation Timestamp (Bytes 16-23):**
  80 aa fa b1 4f 69 dc 01
  This 8-byte segment represents a standard Windows FILETIME structure, indicating the exact date and time this PEK was generated by the primary Domain Controller.
- **Internal Flags & Structural Alignment (Bytes 24-31):**
  00 00 00 00 01 00 00 00 00 00 00 00
  These bytes act as padding and structural markers, defining the algorithm type and length of the subsequent key material.
- **Actual Decrypted PEK (Bytes 32-47):**
  f1 96 9c 98 db ee ca 91 c8 21 33 ac ce 76 3a 39
  By offsetting exactly 32 bytes into the decrypted structure, we locate the raw 16-byte (128-bit) symmetric key.
- **Trailing Padding (Bytes 48+):**
  bf f4 b8 d3 be c6 4c 47 bd 1c 47 de
  The remaining bytes are standard cryptographic padding utilized to align the block size for the AES decryption engine.

By manually carving the structure at the 32-byte offset, we successfully isolated the true Password Encryption Key (*f1969c98dbeeca91c82133acce763a39*). This key represents the master symmetric key for the database and is now staged for the final step: decrypting the individual user NTLM hashes stored within the ntds.dit user objects.